

```

1  import components.naturalnumber.NaturalNumber;
2
3  9
4
5  10 /**
6   * Utilities that could be used with RSA cryptosystems.
7   *
8   * @author Eric Zhou
9   *
10  */
11 public final class CryptoUtilities {
12
13     /**
14      * Private constructor so this utility class cannot be instantiated.
15      */
16     private CryptoUtilities() {
17     }
18
19     /**
20      * Useful constant, not a magic number: 3.
21      */
22     private static final int THREE = 3;
23
24     /**
25      * Pseudo-random number generator.
26      */
27     private static final Random GENERATOR = new Random1L();
28
29     /**
30      * Returns a random number uniformly distributed in the interval [0, n].
31      *
32      * @param n
33      *         top end of interval
34      * @return random number in interval
35      * @requires n > 0
36      * @ensures <pre>
37      *   randomNumber = [a random number uniformly distributed in [0, n]]
38      * </pre>
39      */
40     public static NaturalNumber randomNumber(NaturalNumber n) {
41         assert !n.isZero() : "Violation of: n > 0";
42         final int base = 10;
43         NaturalNumber result;
44         int d = n.divideBy10();
45         if (n.isZero()) {
46             /*
47              * Incoming n has only one digit and it is d, so generate a random
48              * number uniformly distributed in [0, d]
49              */
50             int x = (int) ((d + 1) * GENERATOR.nextDouble());
51             result = new NaturalNumber2(x);
52             n.multiplyBy10(d);
53         } else {
54             /*
55              * Incoming n has more than one digit, so generate a random number
56              * (NaturalNumber) uniformly distributed in [0, n], and another
57              * (int) uniformly distributed in [0, 9] (i.e., a random digit)
58              */
59             result = randomNumber(n);
60             int lastDigit = (int) (base * GENERATOR.nextDouble());
61             result.multiplyBy10(lastDigit);
62             n.multiplyBy10(d);
63             if (result.compareTo(n) > 0) {
64                 /*
65                  * In this case, we need to try again because generated number
66                  * is greater than n; the recursive call's argument is not
67                  * "smaller" than the incoming value of n, but this recursive

```

```

73         * call has no more than a 90% chance of being made (and for
74         * large n, far less than that), so the probability of
75         * termination is 1
76         */
77         result = randomNumber(n);
78     }
79 }
80 return result;
81 }
82
83 /**
84  * Finds the greatest common divisor of n and m.
85  *
86  * @param n
87  *         one number
88  * @param m
89  *         the other number
90  * @updates n
91  * @clears m
92  * @ensures n = [greatest common divisor of #n and #m]
93  */
94 public static void reduceToGCD(NaturalNumber n, NaturalNumber m) {
95
96     /*
97      * Use Euclid's algorithm; in pseudocode: if  $m = 0$  then  $GCD(n, m) = n$ 
98      * else  $GCD(n, m) = GCD(m, n \bmod m)$ 
99      */
100
101     // TODO - fill in body
102     NaturalNumber temp = new NaturalNumber2(0);
103
104     if (!m.isZero()) { //check whether m is zero
105         temp.copyFrom(n);
106         n.copyFrom(m);
107
108         NaturalNumber result = new NaturalNumber2(temp.divide(m));
109
110         m.copyFrom(result);
111         reduceToGCD(n, m); //recursion
112     } else {
113         m.clear(); //clear the m
114     }
115 }
116
117 /**
118  * Reports whether n is even.
119  *
120  * @param n
121  *         the number to be checked
122  * @return true iff n is even
123  * @ensures isEven =  $(n \bmod 2 = 0)$ 
124  */
125
126 public static boolean isEven(NaturalNumber n) {
127
128     // TODO - fill in body
129
130     NaturalNumber temp = new NaturalNumber2(n);
131     NaturalNumber result = new NaturalNumber2(temp.divide(new NaturalNumber2(2)));
132     return result.isZero(); //check the result
133 }
134
135 /**
136  * Updates n to its p-th power modulo m.
137  *

```

```

138  * @param n
139  *         number to be raised to a power
140  * @param p
141  *         the power
142  * @param m
143  *         the modulus
144  * @updates n
145  * @requires m > 1
146  * @ensures n = #n ^ (p) mod m
147  */
148  public static void powerMod(NaturalNumber n, NaturalNumber p, NaturalNumber m) {
149      assert m.compareTo(new NaturalNumber2(1)) > 0 : "Violation of: m > 1";
150
151      /*
152       * Use the fast-powering algorithm as previously discussed in class,
153       * with the additional feature that every multiplication is followed
154       * immediately by "reducing the result modulo m"
155       */
156
157      // TODO - fill in body
158      final NaturalNumber one = new NaturalNumber2(1);
159      final NaturalNumber two = new NaturalNumber2(2);
160
161      NaturalNumber pCopy = new NaturalNumber2(p);
162      NaturalNumber nCopy = new NaturalNumber2(n);
163
164      if (p.isZero()) {
165          n.setFromInt(1); //when exp = 0, the result will be 1
166      } else if (p.compareTo(one) > 0) {
167          if (isEven(p)) {
168              p.divide(two);
169              powerMod(n, p, m);
170              nCopy.copyFrom(n);
171          } else {
172              n.multiply(nCopy); //decrease exp
173              n.copyFrom(n.divide(m));
174              p.subtract(one);
175              p.divide(two);
176              powerMod(n, p, m);
177          }
178
179          p.copyFrom(pCopy); //restore the element
180          n.multiply(nCopy);
181          n.copyFrom(n.divide(m));
182      }
183  }
184
185  /**
186   * Reports whether w is a "witness" that n is composite, in the sense that
187   * either it is a square root of 1 (mod n), or it fails to satisfy the
188   * criterion for primality from Fermat's theorem.
189   *
190   * @param w
191   *         witness candidate
192   * @param n
193   *         number being checked
194   * @return true iff w is a "witness" that n is composite
195   * @requires n > 2 and 1 < w < n - 1
196   * @ensures <pre>
197   *         isWitnessToCompositeness =
198   *         (w ^ 2 mod n = 1) or (w ^ (n-1) mod n /= 1)
199   * </pre>
200   */
201  public static boolean isWitnessToCompositeness(NaturalNumber w, NaturalNumber n) {
202      assert n.compareTo(new NaturalNumber2(2)) > 0 : "Violation of: n > 2";

```

```

203    assert (new NaturalNumber2(1)).compareTo(w) < 0 : "Violation of: 1 < w";
204    n.decrement();
205    assert w.compareTo(n) < 0 : "Violation of: w < n - 1";
206    n.increment();
207
208    // TODO - fill in body
209    final NaturalNumber one = new NaturalNumber2(1);
210
211    boolean result = false;
212    NaturalNumber deOfw = new NaturalNumber2(0);
213
214    deOfw.copyFrom(w);
215    powerMod(w, new NaturalNumber2(2), n); //condition one
216    if (w.compareTo(one) == 0) {
217        result = true;
218    }
219
220    w.copyFrom(deOfw); //restore the element
221
222    NaturalNumber deOfn = new NaturalNumber2(n);
223    deOfn.decrement();
224    powerMod(w, deOfn, n); //condition two
225    if (w.compareTo(one) != 0) {
226        result = true;
227    }
228
229    w.copyFrom(deOfw);
230    return result;
231 }
232
233 /**
234  * Reports whether n is a prime; may be wrong with "low" probability.
235  *
236  * @param n
237  *      number to be checked
238  * @return true means n is very likely prime; false means n is definitely
239  *         composite
240  * @requires n > 1
241  * @ensures <pre>
242  * isPrime1 = [n is a prime number, with small probability of error
243  *             if it is reported to be prime, and no chance of error if it is
244  *             reported to be composite]
245  * </pre>
246  */
247 public static boolean isPrime1(NaturalNumber n) {
248     assert n.compareTo(new NaturalNumber2(1)) > 0 : "Violation of: n > 1";
249     boolean isPrime;
250     if (n.compareTo(new NaturalNumber2(THREE)) <= 0) {
251         /*
252          * 2 and 3 are primes
253          */
254         isPrime = true;
255     } else if (isEven(n)) {
256         /*
257          * evens are composite
258          */
259         isPrime = false;
260     } else {
261         /*
262          * odd n >= 5: simply check whether 2 is a witness that n is
263          * composite (which works surprisingly well :-))
264          */
265         isPrime = !isWitnessToCompositeness(new NaturalNumber2(2), n);
266     }
267     return isPrime;

```

```

268 }
269
270 /**
271  * Reports whether n is a prime; may be wrong with "low" probability.
272  *
273  * @param n
274  *      number to be checked
275  * @return true means n is very likely prime; false means n is definitely
276  *         composite
277  * @requires n > 1
278  * @ensures <pre>
279  *   isPrime2 = [n is a prime number, with small probability of error
280  *               if it is reported to be prime, and no chance of error if it is
281  *               reported to be composite]
282  * </pre>
283  */
284 public static boolean isPrime2(NaturalNumber n) {
285     assert n.compareTo(new NaturalNumber2(1)) > 0 : "Violation of: n > 1";
286
287     /*
288      * Use the ability to generate random numbers (provided by the
289      * randomNumber method above) to generate several witness candidates --
290      * say, 10 to 50 candidates -- guessing that n is prime only if none of
291      * these candidates is a witness to n being composite (based on fact #3
292      * as described in the project description); use the code for isPrime1
293      * as a guide for how to do this, and pay attention to the requires
294      * clause of isWitnessToCompositeness
295      */
296
297     // TODO - fill in body
298     final NaturalNumber two = new NaturalNumber2(2);
299
300     boolean isPrime;
301     if (n.compareTo(new NaturalNumber2(THREE)) <= 0) { //check 2 & 3
302         isPrime = true;
303     } else if (isEven(n)) {
304         isPrime = false;
305     } else {
306         int counter = 0;
307         isPrime = true;
308
309         final int twenty = 20; //set the candidates
310         while (isPrime && counter <= twenty) {
311             n.subtract(two);
312             NaturalNumber random = randomNumber(n);
313
314             while (random.compareTo(two) < 0) {
315                 random = randomNumber(n);
316             }
317             n.add(two);
318             isPrime = !isWitnessToCompositeness(random, n);
319             counter++;
320         }
321     }
322     return isPrime;
323 }
324
325
326 /**
327  * Generates a likely prime number at least as large as some given number.
328  *
329  * @param n
330  *      minimum value of likely prime
331  * @updates n
332  * @requires n > 1

```

```

333     * @ensures n >= #n and [n is very likely a prime number]
334     */
335     public static void generateNextLikelyPrime(NaturalNumber n) {
336         assert n.compareTo(new NaturalNumber2(1)) > 0 : "Violation of: n > 1";
337
338         /*
339          * Use isPrime2 to check numbers, starting at n and increasing through
340          * the odd numbers only (why?), until n is likely prime
341          */
342
343         // TODO - fill in body
344         final NaturalNumber two = new NaturalNumber2(2);
345         boolean isPrime = false;
346
347         if (isEven(n)) {
348             n.increment();
349         } else {
350             n.add(two); //change to odd
351         }
352
353         while (!isPrime) {
354             if (isPrime2(n)) {
355                 isPrime = true;
356             } else {
357                 n.add(two);
358             }
359         }
360     }
361 }
362
363 /**
364  * Main method.
365  *
366  * @param args
367  *         the command line arguments
368  */
369 public static void main(String[] args) {
370     SimpleReader in = new SimpleReader1L();
371     SimpleWriter out = new SimpleWriter1L();
372
373     /*
374      * Sanity check of randomNumber method -- just so everyone can see how
375      * it might be "tested"
376      */
377     final int testValue = 17;
378     final int testSamples = 100000;
379     NaturalNumber test = new NaturalNumber2(testValue);
380     int[] count = new int[testValue + 1];
381     for (int i = 0; i < count.length; i++) {
382         count[i] = 0;
383     }
384     for (int i = 0; i < testSamples; i++) {
385         NaturalNumber rn = randomNumber(test);
386         assert rn.compareTo(test) <= 0 : "Help!";
387         count[rn.toInt()]++;
388     }
389     for (int i = 0; i < count.length; i++) {
390         out.println("count[" + i + "] = " + count[i]);
391     }
392     out.println(
393         "    expected value = " + (double) testSamples / (double) (testValue + 1));
394
395     /*
396      * Check user-supplied numbers for primality, and if a number is not
397      * prime, find the next likely prime after it

```

```

398      */
399      while (true) {
400          out.print("n = ");
401          NaturalNumber n = new NaturalNumber2(in.nextLine());
402          if (n.compareTo(new NaturalNumber2(2)) < 0) {
403              out.println("Bye!");
404              break;
405          } else {
406              if (isPrime1(n)) {
407                  out.println(n + " is probably a prime number"
408                      + " according to isPrime1.");
409              } else {
410                  out.println(n + " is a composite number" + " according to isPrime1.");
411              }
412              if (isPrime2(n)) {
413                  out.println(n + " is probably a prime number"
414                      + " according to isPrime2.");
415              } else {
416                  out.println(n + " is a composite number" + " according to isPrime2.");
417                  generateNextLikelyPrime(n);
418                  out.println(" next likely prime is " + n);
419              }
420          }
421      }
422
423      /*
424      * Close input and output streams
425      */
426      in.close();
427      out.close();
428  }
429
430 }
431

```